

طراحی و ارزیابی یک زمان‌بند **Soft-Deadline** آگاه از کاربرد برای بارهای کاری هوش مصنوعی در لینوکس با استفاده از

sched_ext و Rust

Design and Evaluation of a Rust-Based Application-Aware Soft-Deadline Scheduler for Artificial Intelligence Workloads Using Linux sched_ext

پروپوزال پروژه یک ترمی

1. بیان مسئله

بارهای کاری هوش مصنوعی معمولاً از وظایفی با ویژگی‌های متفاوت تشکیل می‌شوند. بعضی وظایف، مانند پاسخ‌گویی تعاملی، پردازش درخواست کاربر یا اجرای یک مرحله حساس از زنجیره استنتاج، باید در زمان محدودی تکمیل شوند؛ درحالی‌که وظایفی مانند پردازش دسته‌ای داده، ایجاد نمایه یا تولید گزارش می‌توانند با اولویت پایین‌تری اجرا شوند. اجرای هم‌زمان این وظایف روی یک سیستم با منابع محدود ممکن است موجب افزایش تأخیر، از دست رفتن مهلت و محروم‌شدن وظایف کم‌اولویت از پردازنده شود.

زمان‌بند Fair لینوکس در نسخه‌های جدید بر مبنای EEVDF عمل می‌کند. این زمان‌بند برای حفظ عدالت از Virtual Runtime، Lag و Virtual Deadline استفاده می‌کند؛ اما Virtual Deadline آن بیانگر مهلت واقعی اعلام‌شده از سوی برنامه نیست [1]. از سوی دیگر، کلاس SCHED_DEADLINE پیاده‌سازی EDF همراه با Constant Bandwidth Server است و بر اساس پارامترهای runtime/deadline/period پهنای باند CPU را رزرو و کنترل پذیرش انجام می‌دهد. این سازوکار برای رزرو زمانی مناسب است، اما امکان ترکیب آزاد Deadline تخمینی، اولویت سطح کاربرد، Aging، کلاس‌های کاری و تصمیم‌های ADMIT/DELAY/REJECT مبتنی بر فشار سیستم را فراهم نمی‌کند [2].

زیرساخت sched_ext از Linux 6.12 در هسته اصلی موجود است و امکان بارگذاری پویای سیاست‌های زمان‌بندی BPF را بدون Patch کردن مستقیم هسته فراهم می‌کند. در صورت بروز خطا یا توقف یک وظیفه Runnable، زمان‌بند سفارشی غیرفعال و رفتار پیش‌فرض بازبازی می‌شود. با این حال، API‌های آن تضمین پایداری بین نسخه‌های کرنل ندارند [3].

در اکوسیستم sched_ext، زمان‌بند scx_rustland سیاست اصلی را در فضای کاربر و با Rust اجرا می‌کند. این زمان‌بند برای بهبود پاسخ‌گویی، بارهای تعاملی را نسبت به پردازش‌های CPU-intensive پس‌زمینه در اولویت قرار می‌دهد؛ اما Deadline آن از رفتار اجرایی پردازش استنتاج می‌شود و یک مهلت واقعی برای تکمیل درخواست نیست. چارچوب scx_rustland_core نیز رابط‌هایی مانند dispatch_task، dequeue_task و notify_complete را برای ساخت زمان‌بندهای فضای کاربر فراهم می‌کند [4].

مسئله اصلی این پژوهش آن است که آیا استفاده از Deadline صریح سطح کاربرد، تخمین زمان اجرای باقی‌مانده، کلاس وظیفه، Aging و کنترل پذیرش می‌تواند نرخ از دست رفتن مهلت را نسبت به Deadline‌های مجازی و سیاست‌های عمومی موجود کاهش دهد، بدون آنکه سربار یا بی‌عدالتی غیرقابل‌قبولی ایجاد کند.

2. ارتباط با Cloud و Massive Data Fusion

این پروژه یک Cloud Scheduler در سطح Cluster و جایگزین Kubernetes نیست؛ بلکه در سطح هر گره لینوکسی Cloud یا Edge عمل می‌کند. Orchestrator تعیین می‌کند یک سرویس روی کدام گره اجرا شود، اما پس از استقرار سرویس،

زمان‌بند هسته مشخص می‌کند کدام Thread در چه زمانی CPU دریافت کند. بنابراین، سیاست پیشنهادی یک سازوکار **Node-level Resource Management** برای کاهش Tail Latency و حفظ کیفیت سرویس بارهای AI است. تجربه عملی Meta نیز نشان داده است که یک سیاست اختصاصی مبتنی بر sched_ext می‌تواند در یک سرویس Cloud بزرگ، تأخیر P99 مرحله بازیابی را کاهش دهد [8].

در سامانه‌های Massive Data Fusion، جریان‌های داده متعدد از مراحل دریافت، پیش‌پردازش، استخراج و ویژگی، هم‌ترازی، Fusion و تولید تصمیم عبور می‌کنند. این مراحل رفتار و اهمیت یکسانی ندارند: دریافت داده ممکن است I/O-bound، استخراج و ویژگی CPU-bound، مرحله Fusion حافظه‌بر و تولید هشدار دارای Deadline کوتاه باشد. زمان‌بند عمومی از جایگاه هر پردازش در Pipeline آگاه نیست؛ درحالی‌که سیاست پیشنهادی می‌تواند Deadline و کلاس هر مرحله را دریافت و مرحله حساس تولید تصمیم را بر Indexing یا پردازش Batch مقدم کند.

مرز دقیق پژوهش

این پروژه الگوریتم جدید Data Fusion یا زمان‌بندی چندگره‌ی Cloud طراحی نمی‌کند؛ بلکه زیرساخت اجرای قابل‌پیش‌بینی مراحل یک Pipeline همجوشی داده را در سطح سیستم‌عامل هر گره فراهم می‌کند. برای نشان‌دادن این ارتباط، مطالعه موردی نهایی به صورت یک Pipeline سبک شامل دریافت داده، پیش‌پردازش، استخراج و ویژگی، Fusion و تولید تصمیم اجرا خواهد شد.

3. هدف اصلی و اهداف فرعی

هدف اصلی پروژه، طراحی و ارزیابی یک سیاست Soft-Deadline آگاه از کاربرد برای بارهای کاری ناهمگن هوش مصنوعی است که منطق آن با Rust در فضای کاربر اجرا شده و از طریق scx_rustland_core و sched_ext بر زمان‌بندی واقعی CPU اثر بگذارد.

1. ساخت تولیدکننده بار کاری Config-driven با Rust برای سناریوهای تکرارپذیر؛

2. تعریف فراداده سطح کاربرد برای وظایف و مراحل Pipeline؛

3. پیاده‌سازی سیاست مبتنی بر Laxity، Application Deadline و Aging؛

4. طراحی کنترل پذیرش ساده و PSI-aware برای شرایط اضافه‌بار؛

5. اندازه‌گیری سربار تصمیم‌گیری در فضای کاربر و ارتباط با مؤلفه BPF؛

6. مقایسه با EEVDF، SCHED_DEADLINE و scx_rustland استاندارد؛

7. بررسی جداگانه اثر Aging، Application Deadline و Admission Control؛

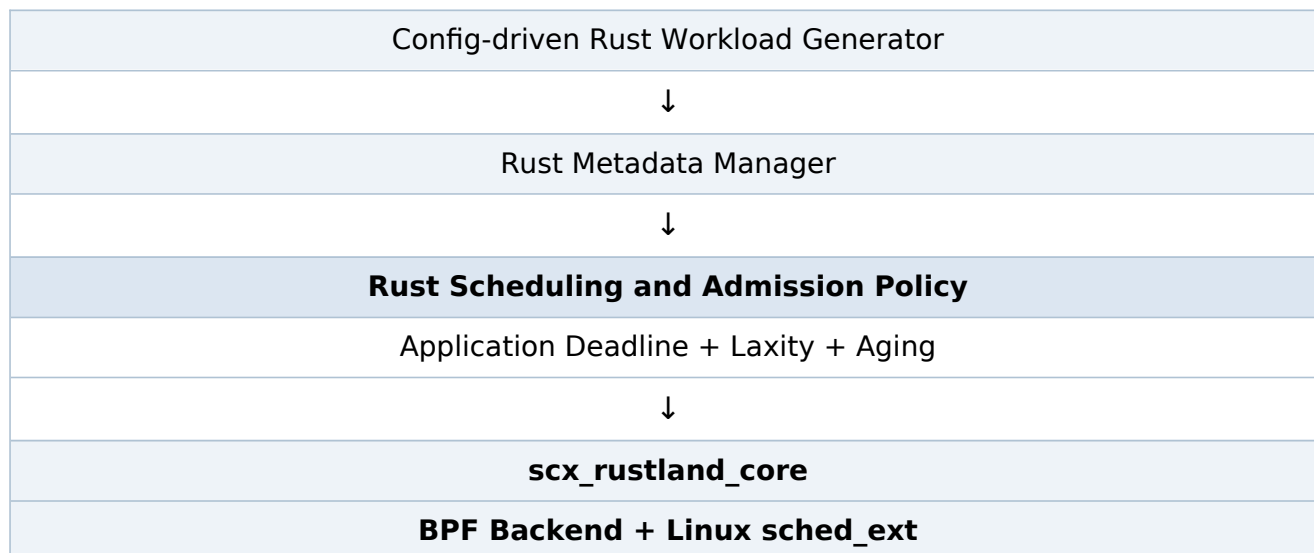
8. ارزیابی Deadline Miss، Tail Latency، Throughput، Fairness و Scheduler Overhead.

4. سؤالات پژوهش

سؤال اصلی: آیا استفاده از Deadline صریح سطح کاربرد و زمان اجرای باقی‌مانده، در مقایسه با Deadline‌های مجازی و استنتاج‌شده در زمان‌بندهای موجود، نرخ از دست رفتن مهلت بارهای کاری ناهمگن AI و Data Fusion را کاهش می‌دهد؟

- سربار اجرای سیاست در فضای کاربر و ارتباط آن با BPF چقدر است؟
- Aging چه تأثیری بر عدالت و جلوگیری از گرسنگی وظایف Batch دارد؟
- کنترل پذیرش چه تأثیری بر Deadline Miss و Tail Latency در شرایط اضافه بار دارد؟
- روش پیشنهادی در چه نوع بارهایی نسبت به SCHED_DEADLINE یا scx_rustland مناسب تر است؟

5. معماری پیشنهادی



برای کاهش ریسک فنی، پل Rust و BPF از ابتدا ساخته نمی شود و پروژه بر پایه scx_rustland_core توسعه می یابد. این انتخاب تغییر سریع سیاست را ممکن می کند، اما هزینه انتقال تصمیم میان فضای کاربر و هسته را به همراه دارد. اندازه گیری همین هزینه بخشی از ارزیابی است.

6. فراداده وظایف و مراحل Pipeline

شرح	فیلد
شناسه یکتای وظیفه	task_id
شناسه پردازش یا Thread	process_id
شناسه گردش کار یا Pipeline همجوشی داده	workflow_id
شناسه مرحله در Pipeline	stage_id
یکی از کلاس های critical, interactive یا batch	workload_class
مهلت مطلق تکمیل وظیفه	absolute_deadline
زمان اجرای باقی مانده تخمینی	estimated_remaining_runtime
اولویت اعلام شده از سوی کاربر	application_priority
زمان ورود وظیفه به سامانه	release_time

7. تخمین زمان اجرا و سیاست زمان‌بندی

برای حفظ دامنه یک‌ترمی، از مدل یادگیری ماشین برای تخمین زمان اجرا استفاده نمی‌شود. زمان اجرای پایه هر نوع وظیفه از چند اجرای کالبراسیون به دست می‌آید و با میانگین متحرک ساده اصلاح می‌شود. زمان اجرای باقی‌مانده از تفاضل تخمین کل و CPU Time مصرف‌شده محاسبه خواهد شد:

$$\hat{R}_i(t) = \max(0, \hat{C}_i - C_{i,consumed}(t))$$

در این رابطه $\hat{C}_i$ تخمین زمان اجرای کل وظیفه و $C_{i,consumed}(t)$ زمان CPU مصرف‌شده تا لحظه t است. برای هر وظیفه دارای مهلت، Laxity به صورت زیر محاسبه می‌شود:

$$L_i(t) = D_i - t - \hat{R}_i(t)$$

مقدار کمتر Laxity نشان‌دهنده فوریت بیشتر است. امتیاز نهایی با ترکیب Laxity، اولویت کاربرد و Aging تعیین می‌شود:

$$S_i(t) = L_i(t) - \alpha P_i^* - \beta A_i^*(t)$$

در این فرمول P_i^* و $A_i^*(t)$ نسخه‌های نرمال‌شده اولویت و Aging هستند. ضرایب α و β با واحد زمانی تعریف می‌شوند تا جمع مؤلفه‌ها از نظر ابعادی معتبر باشد. وظیفه دارای کمترین $S_i(t)$ در اولویت Dispatch قرار می‌گیرد. هدف پروژه ارائه تضمین Hard Real-Time نیست؛ بلکه بهبود Soft Deadline و کیفیت سرویس است.

8. کنترل پذیرش و فشار CPU

کنترل پذیرش دارای سه خروجی ADMIT، DELAY و REJECT خواهد بود. تصمیم بر اساس زمان باقی‌مانده تا Deadline، زمان اجرای تخمینی، تعداد وظایف فعال و فشار CPU اتخاذ می‌شود. PSI کمبود منابع و زمان توقف ناشی از رقابت را گزارش می‌کند [7]. در این پروژه فقط رابط فشار CPU و شاخص some avg10 استفاده خواهد شد تا دامنه کنترل شود.

/proc/pressure/cpu

کنترل پذیرش این پروژه با کنترل پذیرش SCHED_DEADLINE متفاوت است: SCHED_DEADLINE بر امکان رزرو پهنای باند با پارامترهای runtime/deadline/period تمرکز دارد؛ درحالی‌که کنترل‌کننده پیشنهادی می‌تواند یک وظیفه Soft-Deadline را بر اساس فشار جاری سیستم و اهمیت کاربرد بپذیرد، به تأخیر بیندازد یا رد کند.

9. تولیدکننده بار کاری و مطالعه موردی

تولیدکننده بار کاری با Rust نوشته می‌شود و از فایل تنظیمات، سناریوهای سبک، متوسط، سنگین و انفجاری را اجرا می‌کند. بارها شامل وظایف CPU-bound، Memory-bound و I/O-bound با زمان ورود، Deadline و مدت اجرای متفاوت خواهند بود. Config-driven بودن ابزار سبب می‌شود Baseline‌ها و Ablation‌ها بدون تغییر کد و فقط با تغییر فایل تنظیمات اجرا شوند.

پس از تکمیل هسته اصلی، یک مطالعه موردی سبک از Pipeline همجوشی داده شامل دریافت داده، پیش‌پردازش، استخراج ویژگی، Fusion و تولید تصمیم روی یک گره Linux Cloud/Edge اجرا می‌شود. مراحل Pipeline به صورت وظایف ناهمگن با Deadline و اولویت متفاوت مدل می‌شوند. این مطالعه صرفاً برای ارزیابی زمان‌بند است و طراحی الگوریتم جدید Fusion یا زمان‌بندی چندگره‌ای در محدوده پروژه نیست.

10. نوآوری و سهم پژوهش

این پروژه الگوریتم EDF، Laxity یا Aging جدیدی معرفی نمی‌کند. سهم پژوهش در طراحی و ارزیابی تجربی یک سیاست Application-Aware و مشخص کردن محدوده‌ای است که Deadline معنایی کاربرد نسبت به Deadline مجازی مزیت ایجاد می‌کند.

1. استفاده از Deadline واقعی و اعلام شده توسط برنامه، به جای اتکا به Virtual Deadline استنتاج شده از رفتار پردازش؛
 2. ترکیب Application Deadline، تخمین زمان باقی مانده، کلاس وظیفه، Aging و Admission Control در یک سیاست Soft-Deadline؛
 3. پیاده سازی سیاست با Rust و بدون تغییر مستقیم کد هسته؛
 4. مقایسه تجربی با نزدیک ترین روش ها، به ویژه scx_rustland و SCHED_DEADLINE؛
 5. اندازه گیری هزینه تصمیم گیری در فضای کاربر به ازای Dispatch و چرخه ارتباطی؛
 6. ارزیابی یک Pipeline سبک Massive Data Fusion در سطح یک گره Cloud/Edge.
- در کارهای مرتبط، scx_rusty به عنوان معماری هیبریدی و scx_lavd به عنوان سیاستی که تصمیم های اصلی آن در BPF انجام می شود بررسی خواهند شد [5][6]. اجرای scx_lavd به عنوان Baseline کامل، جزء الزامی پروژه یک ترمی نیست. تجربه Meta نیز اهمیت صنعتی زمان بندی آگاه از کاربرد را نشان می دهد: این شرکت یک سیاست اختصاصی مبتنی بر sched_ext را برای سرویس تبلیغات خود گزارش کرده است؛ منبع رسمی این سیاست را scx_lavd معرفی نمی کند [8].

11. روش ارزیابی

چهار حالت اصلی در سناریوهای قابل اعمال مقایسه می شوند:

روش	نقش در ارزیابی
EEVDF	زمان بند Fair پیش فرض لینوکس؛
SCHED_DEADLINE	فقط برای سناریوهای Periodic یا Sporadic که تعریف منصفانه runtime/deadline/period برای آن ها ممکن باشد؛ در سایر سناریوها «قابل اعمال نیست» گزارش می شود.
scx_rustland	نسخه استاندارد بدون تغییر، به عنوان نزدیک ترین Baseline مبتنی بر Deadline مجازی؛
روش پیشنهادی	Application Deadline + Laxity + Aging + Admission Control.

سه Ablation فقط در سناریوهای منتخب اجرا خواهند شد:

- روش پیشنهادی بدون Aging؛
- روش پیشنهادی بدون Admission Control؛
- روش پیشنهادی بدون Application Deadline و با جایگزینی آن توسط Virtual Deadline مشابه scx_rustland.

معیارهای اصلی شامل Deadline Miss Ratio، میانگین و صدک 95 زمان پاسخ، CPU Utilization، Throughput، Jain's Fairness Index و زمان CPU مصرفی Scheduler فضای کاربر است. برای سنجش سربار scx_rustland_core، اصطلاحات و شمارنده‌های خود API شامل User-space dispatches، Kernel dispatches، Cancelled dispatches، Bounced dispatches، Failed dispatches و Scheduler congestion events گزارش می‌شوند [4]. همچنین متوسط تعداد وظایف پردازش‌شده در هر چرخه notify_complete ثبت خواهد شد.

هر آزمایش حداقل 10 بار تکرار می‌شود و میانگین، انحراف معیار و فاصله اطمینان 95 درصد گزارش خواهد شد. فقط چهار روش اصلی روی همه شدت‌های بار اجرا می‌شوند و Ablation ها به سناریوهای منتخب محدود خواهند بود.

12. محیط اجرا، محدوده و کنترل ریسک

تمام آزمایش‌ها روی Linux 6.12.95 انجام می‌شوند. به دلیل ناپایداری ABI مربوط به sched_ext، در هفته نخست یک Tag یا Commit سازگار از مخزن scx انتخاب، SHA آن قفل و در کد، فایل تنظیمات و گزارش نهایی ثبت خواهد شد. تغییر نسخه کرنل یا scx در میانه آزمایش‌ها مجاز نخواهد بود [3].

پروژه فقط زمان‌بندی CPU روی یک گره را بررسی می‌کند. GPU Scheduling، Kubernetes، زمان‌بندی چندگره‌ای، Edge-Cloud Offloading، یادگیری تقویتی و Hard Real-Time خارج از محدوده هستند. بزرگ‌ترین ریسک، راه‌اندازی صحیح sched_ext و scx_rustland_core است؛ بنابراین در دو هفته نخست ابتدا scx_simple و سپس scx_rustland بدون تغییر اجرا خواهند شد. اگر سربار Rustland زیاد باشد، این موضوع به عنوان نتیجه پژوهش گزارش و انتقال Hot Path به BPF به کار آینده واگذار می‌شود.

13. برنامه زمان‌بندی یک‌ترمی

فعالیت	هفته
اجرای scx_simple و scx_rustland، انتخاب Commit، ثابت، تثبیت محیط و اندازه‌گیری اولیه	1 و 2
پیاده‌سازی Workload Generator و Metadata Manager	3 و 4
پیاده‌سازی Application Deadline، تخمین Runtime و محاسبه Laxity	5 و 6
افزودن Aging و تکمیل سیاست Dispatch	7 و 8
پیاده‌سازی Admission Control و PSI	9
تکمیل ابزار ثبت معیارها و فایل‌های تنظیمات	10
اجرای Baseline ها و آزمایش‌های اصلی	11 و 12
اجرای Ablation و تحلیل نتایج	13
نگارش گزارش، نمودارها و آماده‌سازی ارائه	14

1. تولیدکننده بار کاری Config-driven با Rust؛
2. مدیر فراداده وظایف و مراحل Pipeline؛
3. سیاست زمان‌بندی Application-Aware مبتنی بر Rust؛
4. کنترل پذیرش ساده و PSI-aware؛
5. اتصال عملی به scx_rustland_core و sched_ext؛
6. ابزار جمع‌آوری نتایج و خروجی CSV؛
7. اسکریپت اجرای خودکار آزمایش‌ها؛
8. مقایسه با EEVDF، SCHED_DEADLINE و scx_rustland؛
9. مطالعه موردی سبک Massive Data Fusion روی یک گره Linux Cloud/Edge؛
10. گزارش نهایی، جداول، نمودارها و راهنمای اجرا.

15. منابع اولیه

- [1] Linux Kernel Documentation, "EEVDF Scheduler." <https://docs.kernel.org/scheduler/sched-eevdf.html>
- [2] Linux Kernel Documentation, "Deadline Task Scheduling." <https://docs.kernel.org/scheduler/sched-deadline.html>
- [3] Linux Kernel Documentation, "Extensible Scheduler Class - sched_ext," Linux 6.12. <https://docs.kernel.org/6.12/scheduler/sched-ext.html>
- [4] sched-ext/scx, "scx_rustland_core" and "scx_rustland" documentation. https://github.com/sched-ext/scx/blob/main/rust/scx_rustland_core/README.md
- [5] sched-ext/scx, "scx_rusty" documentation. https://github.com/sched-ext/scx/blob/main/scheds/rust/scx_rusty/README.md
- [6] sched-ext/scx, "scx_lavd: Latency-criticality Aware Virtual Deadline." https://github.com/sched-ext/scx/blob/main/scheds/rust/scx_lavd/README.md
- [7] Linux Kernel Documentation, "PSI - Pressure Stall Information." <https://docs.kernel.org/accounting/psi.html>
- [8] Meta Engineering, "Modernizing the Meta Ads Service With an Open-Source Kernel Scheduler," July 13, 2026. <https://engineering.fb.com/2026/07/13/ml-applications/modernizing-the-meta-ads-service-with-an-open-source-kernel-scheduler/>

تاریخ بررسی منابع: 11 مرداد 1405